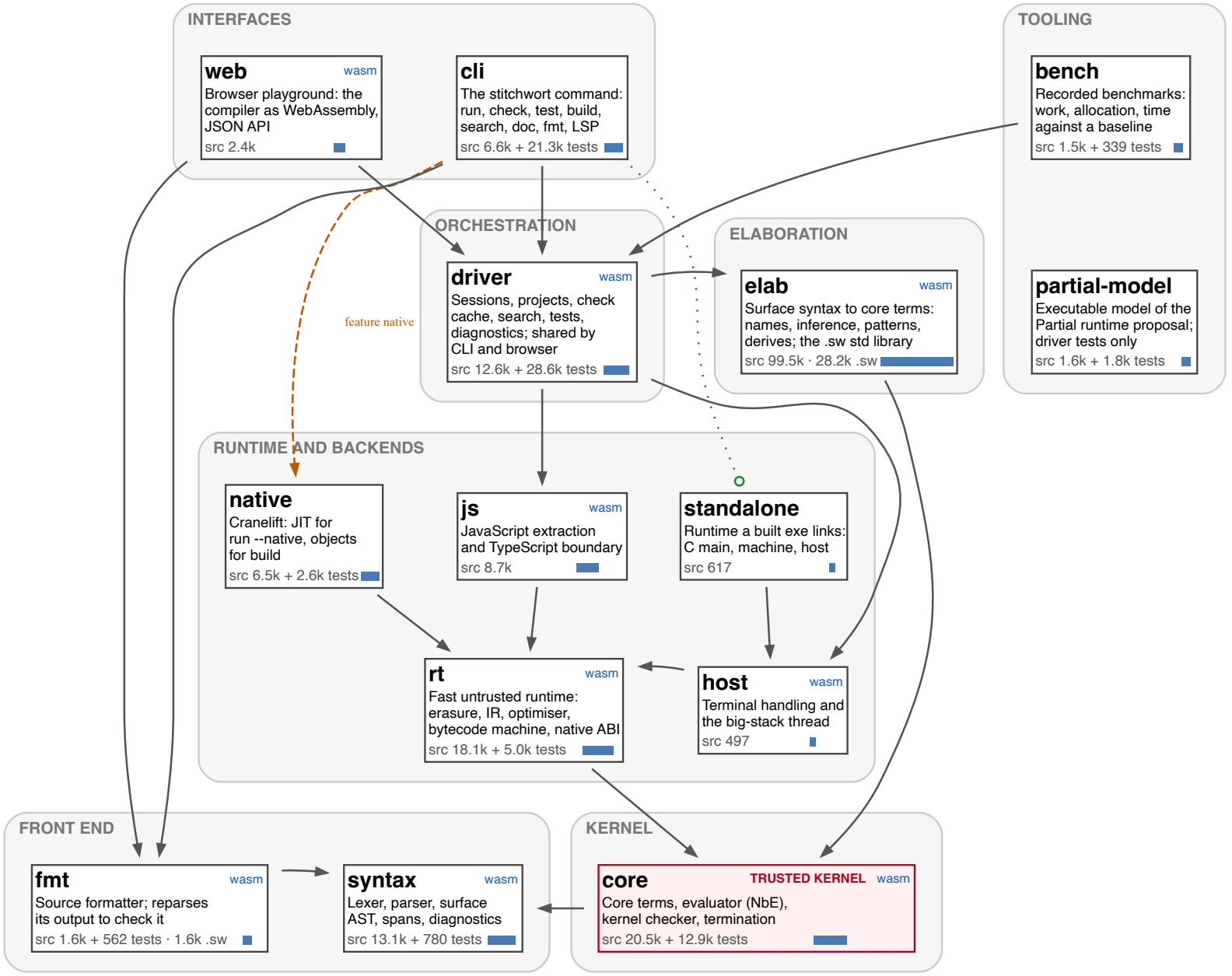
Stitchwort: the crates

workspace at commit 5cc0e13c 2026-11-03 · generated from `cargo metadata` and the sources



How to read it

- An arrow `A → B` means A depends on B. Layers run from the interfaces (top) down to the front end. Edges implied by a longer path are left out (`cli → core` is there through `driver`), so the picture shows structure, not every `use`.
- Dashed orange: optional dependency (the CLI's `native` feature).
- Dotted green: not a Cargo edge. `stitchwort build` links `standalone`'s static library; `cli` has it only as a dev-dependency, so tests build it.
- wasm: part of the browser playground's build (everything `web` pulls in, through `driver`).
- Red box: the trusted kernel.
- Sizes: `src` is Rust lines in `src/`, `tests` the rest of each crate's Rust (tests, examples, benches), `.sw` Stitchwort lines. The blue bar is `src`, on a square-root scale.
- Dev-dependencies are not drawn. They mostly point at `driver`, `core` and `rt`, for tests.

Boundary facts

- `core` is the kernel and more: it also has the pretty-printer, a reference evaluator for differential tests (feature `reference`), erasure, content hashing, resource budgets, and `Term::Meta`, the elaborator's metavariable.
- `rt` holds most of the backend: erasure, IR, optimiser, bytecode machine, native ABI. `js` and `native` build on it, and `host` and `standalone` run it.
- The runtimes (`rt`, `js`, `native`, `standalone`) depend on `core` and `syntax`, never on `elab`: they run checked definitions and are untrusted, so they can get an output wrong but not make a false proof check.
- `elab` is the largest crate by far, and carries the `.sw` standard library (`src/std/`).
- `driver` is the shared pipeline: `cli` and `web` both sit on it, so neither reimplements checking.
- `native` (Cranelift) is optional for the CLI and absent from the WebAssembly build; `host` is in it, pulled in by `driver`.
- `partial-model` is used by nothing in the compiler, only by `driver`'s tests; `bench` measures `driver`.
- Not Rust and not shown: `board/` (ticket board, JS), `editors/vscode`, the playground page in `web/`, and the `.sw` examples and games.